

ML.py에서 해석이 끝난후 gpr학습하는데 시간을 너무 잡아 먹는거같아..

10s 동안 처리함

응. 코드 전체를 보면 “해석(CFD) 끝난 뒤 GPR 학습이 오래 걸리는 문제”가 실제로 코드에 명확히 있습니다. 그리고 현재 버전에는 성능상 꽤 중요한 포인트가 하나 더 보입니다.

가장 큰 병목

현재 적응 샘플링에서 `_gpr_suggest()` 가 호출되면 `_fit_models(df, use_cache=True)` 로 19개 GPR을 학습합니다. 모델 수는 목적함수 17개 + 제약조건 2개입니다. [GitHub +1](#)

그런데 `update_ml()` 에서도 예측 캐시가 없으면 `_predict_point()` 를 호출하고, 여기서 다시 `_fit_models(df)` 를 실행합니다. 즉 정상적인 흐름에서는 중복 학습을 피하도록 만들어놨지만, 캐시가 깨지는 순간 19개 모델을 다시 처음부터 최적화하게 됩니다. [GitHub](#)

더 중요한 건 이것입니다.

Python

실행됨

```
def _fit_models(df, use_cache=False):
    ...
    refresh = (
        (not use_cache)
        or (not _KERNEL_CACHE)
        or (_ROUNDS_SINCE_REFRESH >= _REFRESH_EVERY)
    )
```

`use_cache=False` 이면 무조건 `refresh=True` 입니다. 그리고 `_predict_point()` 는:

Python

실행됨

```
models = _fit_models(df)
```

처럼 캐시를 아예 사용하지 않습니다. [GitHub](#)

그래서 `_PENDING_PREDICTION` 이 정상적으로 작동하면 괜찮지만, 재시작이나 호출 순서가 조금만 달라져도 GPR 19개를 다시 풀로 최적화하게 됩니다.

그런데 캐시 구현 자체는 현재 정상입니다

제가 처음엔 `_KERNEL_CACHE` 저장 위치도 의심했는데 raw 코드를 확인해보니 이 부분은 제대로 되어 있습니다.

Python

실행됨

```
for name, gpr, use_log in fitted:
    models[name] = (gpr, use_log)
    if use_cache and refresh:
        _KERNEL_CACHE[name] = gpr.kernel_
```

즉 19개 모델 각각의 최적화된 kernel을 저장합니다.  GitHub

그래서 현재 정상적인 `get_next_params()` → CFD → `update_ml()` 흐름에서 가장 큰 문제는 캐시 그 자체가 아니라, `refresh`가 발생하는 주기와 GPR 개수, 그리고 IMSE 계산량입니다.

실제로 시간이 많이 걸릴 수 있는 곳

현재 설정이:

Python

실행됨

```
N_CAND      = 65536
N_IMSE_CAND = 2000
N_IMSE_REF  = 1024
```

입니다.  GitHub

그리고 `_gpr_suggest()`에서 19개 중 실제 목적함수 17개에 대해

Python

실행됨

```
gpr.predict(cand_norm, return_std=True)
```

를 최대 **65,536개 후보 × 17 GPR**에 수행합니다. 이후 상위 2,000개 후보에 대해 IMSE를 계산하고, 이것 역시 각 active objective마다 수행합니다.  GitHub

특히 `_imse_reduction()`은

Python

실행됨

```
Kxc = k1(Xt, cand)
A    = cho_solve((gpr.L_, True), Kxc)
chat = ...
```

형태라서 후보 수가 커질수록 행렬 연산량이 빠르게 증가합니다.  GitHub

즉 지금은 사실상:

CFD 완료 → GPR 19개 → 65,536 후보 σ 계산 → 2,000 후보 IMSE × 여러 GPR → 다음 파라미터

라는 꽤 무거운 파이프라인입니다.

제가 보기엔 우선순위가 이렇습니다

1순위: `N_CAND` / `N_IMSE_CAND` 줄이기

현재

Python

실행됨

```
N_CAND = 65536
N_IMSE_CAND = 2000
N_IMSE_REF = 1024
```

인데, 이건 꽤 공격적인 설정입니다.

예를 들어 먼저

Python

실행됨

```
N_CAND = 16384
N_IMSE_CAND = 500
N_IMSE_REF = 512
```

정도로 낮춰보는 게 좋습니다.

이렇게 하면 **GPR의 모델 자체를 바꾸지 않고** 후보 탐색 비용만 크게 줄일 수 있습니다. 특히 8차원 설계공간에서는 Sobol 65k → 16k가 꼭 필요한 수준인지 의문입니다.

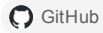
2순위: GPR의 실제 학습 시간과 IMSE 시간을 분리해서 측정

지금 사용자 입장에서는 그냥 “GPR 학습이 오래 걸린다”고 느껴지지만, 실제로는

```
_fit_models()
├─ 19개 GPR fitting
└─ _gpr_suggest()
   ├─ 65k prediction
   └─ IMSE
```

중 어디가 진짜 범인인지 확인해야 합니다.

현재 코드 주석에는 예전에 **83점/19개 모델에서 322.9초 → 약 4초까지 줄었다고** 적혀 있습니다. 즉 작성자 본인도 GPR 재학습을 주요 병목으로 보고 이미 상당한 최적화를 넣어둔 상태입니다.



따라서 지금 실제 실행에서 여전히 수십 초~수분씩 걸린다면, 지금은 **GPR fitting** 자체보다 **IMSE** 쪽일 가능성도 충분히 있습니다.

3순위: `_REFRESH_EVERY = 5` 재검토

현재 5회마다 19개 모델의 hyperparameter optimization을 다시 합니다.

이 순간은 일반적인 `warm_kernel` 경로보다 훨씬 무겁습니다.

Python

실행됨

```
warm = None if refresh else _KERNEL_CACHE.get(name)
```

`refresh=True` 인 회차에는 결국:

Python

실행됨

```
GaussianProcessRegressor(  
    ...,  
    n_restarts_optimizer=3  
)
```

로 다시 최적화합니다.

여기서 데이터가 이미 100~150점 정도 쌓였다면 이 비용이 다시 커질 수 있습니다.

그리고 아주 중요한 구조적 개선이 하나 있습니다

현재는 `get_next_params()` 에서 GPR을 학습한 뒤 `_PENDING_PREDICTION` 에 다음 점의 예측 값만 저장합니다.

즉 이 캐시는:

모델 자체는 저장하지 않고
예측 결과만 저장

하는 구조입니다.

그래서 다음 `update_ml()`에서는 예측만 재사용할 뿐이고, 다음 `get_next_params()` 호출 때 데이터가 하나 추가되면 19개 GPR을 다시 fit해야 합니다.

이건 GPR 특성상 어느 정도 unavoidable하지만, 매번 새 데이터가 1개 추가될 때마다 19개 GP를 전부 다시 계산하는 구조 자체를 더 개선할 여지가 있습니다.

제가 추천하는 방향

지금 코드 구조를 크게 뜯어고치지 않고도 다음 정도로 바꾸면 좋습니다.

현재

```
CFD
↓
19 GPR fit
↓
65,536 candidate prediction
↓
2,000 IMSE
↓
next point
```

예

```
CFD
↓
19 GPR fit
↓
16,384 candidate prediction
↓
500 IMSE
↓
next point
```

로 먼저 줄이고,

그 다음

```
5회마다 full hyperparameter optimization
↓
10~15회마다 optimization
```

처럼 refresh 빈도를 테스트합니다.

그리고 가장 좋은 방법은 코드에 시간 측정을 넣어서:

```
[GPR FIT]      3.8 sec
[GPR PREDICT]   1.7 sec
[IMSE]          12.4 sec
[CSV SAVE]      0.1 sec
[TOTAL]         18.0 sec
```

이렇게 각 구간을 실제로 짚어보는 것입니다.

현재 코드만 놓고 보면 저는 “GPR 학습 자체가 무조건 문제”라고 단정하지 않고,

`_fit_models()` 와 `_gpr_suggest()` 를 분리해서 벤치마크하는 것이 가장 먼저라고 봅니다. 코드상 이미 GPR 재학습은 상당히 최적화되어 있고, 반대로 65k 후보 + 2k IMSE라는 상당히 무거운 탐색이 남아 있기 때문입니다. [GitHub](#)

원하면 제가 다음 답변에서 ** 이 `ML.py` 를 기준으로 실제로 시간을 확 줄이는 수정안을 함수 단위로 바로 제시**할 수 있습니다.

16,384 candidate prediction ↓ 500 IMSE 이걸 안될듯 ..
19개 gpr fit을 줄이는 방법없나 내가 이걸 5개에서 19개로 늘려서 체감시간이 확늘어난거거든 ..

맞아. 그렇다면 candidate/IMSE는 건드리지 말고, 19개 GPR fit 자체를 줄이는 방향으로 가는 게 맞아.

지금 코드를 다시 보면, 19개를 늘린 뒤 시간이 확 뛴 이유가 정확히 설명돼. 현재 `MODELED` 가 17개 **objective + 2개 constraint = 19개**이고, `_fit_models()` 가 이 19개를 각각 별도의 `GaussianProcessRegressor.fit()` 으로 돌린다. [GitHub +1](#)

그런데 여기서 중요한 건:

19개를 꼭 각각 독립적으로 hyperparameter optimization 할 필요가 있느냐?

나는 아니라고 봐.

가장 효과적인 방법: "19개 GPR → 일부만 hyperparameter 최적화"

현재 각 GPR은

Python

실행됨

```
ConstantKernel
*
Matern(length_scale=[...8개...])
```

+

WhiteKernel

이고, 각각 ARD length scale 8개를 따로 최적화한다. 즉 19개 $\times$ (1 + 8 + noise) 계열의 hyperparameter를 각각 최적화하고 있다. [GitHub](#)

특히 **lane 14개**는 서로 완전히 독립적인 물리량이 아니다.

현재:

```
pass1 lane1
pass1 lane2
...
pass1 lane7
pass2 lane1
...
pass2 lane7
```

을 모두 별도 GPR로 학습한다. [GitHub](#)

여기서 내가 가장 먼저 해볼 것은 **lane GPR의 kernel hyperparameter**를 공유하는 것이야.

예를 들어:

```
pressure_drop    → 개별 kernel 최적화
temp_std         → 개별 kernel 최적화
max_temp         → 개별 kernel 최적화

lane 1~7 pass1   → 하나의 공통 kernel
lane 1~7 pass2   → 하나의 공통 kernel

constraint       → 개별 또는 고정
```

그러면 19개 모두 optimizer를 돌리는 대신 실제로는 대략 **7~9번 정도만 hyperparameter optimization**하고, 나머지는 그 kernel을 그대로 사용해서 GP를 fit하는 식으로 만들 수 있어.

더 공격적으로 줄이면

사실 현재 코드에 이미 아주 좋은 장치가 있어.

Python

실행됨

```
warm_kernel = _KERNEL_CACHE.get(name)
```

그리고 `optimizer=None`으로 kernel을 고정한 뒤 `gpr.fit()` 만 한다. [GitHub](#)

즉:

kernel hyperparameter 탐색

↓

19개 모델 각각 optimizer

이게 비싸고,

이미 좋은 kernel 있음

↓

optimizer=None

↓

GPR fit

은 훨씬 싸다.

따라서 **5회마다 19개 전부 재최적화하는 현재 방식보다 한 단계 더 나아가서, kernel을 재최적화하는 모델을 아주 적게 만들 수 있다.**

내가 추천하는 구조

현재:

19 GPR

회차 1:

19개 optimizer + fit

회차 2:

19개 warm fit

회차 3:

19개 warm fit

회차 4:

19개 warm fit

회차 5:

19개 optimizer + fit

를

대표 GPR 몇 개:

kernel optimizer + fit

나머지 GPR:

공유된 kernel + optimizer=None + fit

으로 바꾸는 거야.

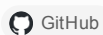
예를 들어:

```
pressure_drop      → optimize
temp_std           → optimize
max_temp          → optimize
pass1 lanes        → 대표 1개 optimize
pass2 lanes        → 대표 1개 optimize
power_module_flow → optimize
weight             → optimize
```

나머지 12개 lane GPR → kernel 재사용

그러면 **19번의 hyperparameter optimization**이 7번 정도로 떨어진다.

그리고 새 데이터가 들어왔을 때에도 각 GP의 covariance matrix / Cholesky decomposition은 다시 계산해야 하니까 **예측 정확성은 유지하면서 optimizer 비용만 크게 줄일 수 있다**. 현재 코드도 warm kernel을 사용할 때 새 데이터에 대한 `gpr.fit()` 은 다시 수행하도록 되어 있다.



그런데 더 좋은 방법이 하나 있음

Multi-output GPR을 고려할 수도 있어.

즉 현재:

```
X → y1 (pressure)
X → y2 (temp_std)
X → y3 (max_temp)
...
X → y19
```

를 하나의 multi-output 모델로 만드는 방식.

다만 나는 지금 바로 이 방향으로 가는 건 추천하지 않아.

scikit-learn의 `GaussianProcessRegressor` 는 여기서 우리가 원하는 형태의 출력 간 상관관계를 적극적으로 이용하는 **multi-task GP**로 바로 바꾸는 게 간단하지 않고, 구현 난이도와 검증량이 확 올라간다.

지금 코드가 이미 잘 돌아가고 있고 CFD 자체가 비싼 상황이라면, 굳이 모델 구조를 크게 바꿀 필요가 없어.

오히려 가장 현실적인 최적화

내가 이 코드라면 이렇게 바꿀 것 같아.

1. 19개 모델 중 kernel optimizer는 최초 1회

처음 adaptive sampling 진입 시:

```
19개 × hyperparameter optimization
```

한 번 수행.

그 이후에는:

```
19개 × optimizer=None
```

로 계속 간다.

그러면 `_REFRESH_EVERY = 5` 자체가 없어져.

현재 코드의 가장 비싼 부분은 바로 이 refresh야. 코드상 `refresh=True` 가 되면 `warm=None` 이 되어 각 GPR가 다시 optimizer를 돌게 된다. [GitHub](#)

2. 대신 "kernel이 정말 바뀌었는지"만 가끔 검사

예를 들어 20회마다:

```
19개 전부 재최적화
```

가 아니라

```
대표적인 3~5개만 재최적화
```

해서 kernel이 크게 바뀌었으면 전체 refresh.

이렇게 하면:

평상시:

```
19 × cheap fit
```

refresh:

```
3~5 × expensive optimization
```

이 된다.

3. 특히 lane 14개는 강하게 묶어도 될 가능성이 높음

이게 이번 문제에서 제일 큰 잠재적인 절감 포인트야.

현재 lane 14개는 모두 같은 8개 geometry 변수의 함수이고, 같은 CFD에서 나온 값들이야. 코드 주석에서도 레인 유량을 하나의 물리적 그룹으로 취급하고 있고, 종료판정에서도 7개 lane을 하나의 group으로 묶고 있다. [GitHub](#)

그런데 학습 단계에서는 여전히:

Python

실행됨

```
[(n, False) for n in LANE_NAMES]
```

으로 14개의 완전 독립 GPR을 만든다. [GitHub](#)

따라서 공통 **kernel**을 사용해도 되는지 먼저 실험해볼 가치가 굉장히 크다.

예:

```
pass1 lane1 → kernel A
pass1 lane2 → kernel A
...
pass1 lane7 → kernel A

pass2 lane1 → kernel B
...
pass2 lane7 → kernel B
```

여기서 각 lane의 **y**는 당연히 별도로 fit한다.

즉 모델을 합치는 게 아니라 **hyperparameter** 탐색만 공유하는 거야.

이건 정확도 손실 위험이 multi-output GP보다 훨씬 작고 코드 변경도 상대적으로 작다.

그리고 한 가지 더

현재 주석에 이미 이런 실측 결과가 있어:

19개 모델, 83점에서 기존 322.9초 → 4초 안팎

즉 작성자가 이미 **jobjlib 병렬화 + warm kernel + restart 감소 + refresh**를 적용해서 상당히 줄여놓은 상태다. [GitHub](#)

그래서 지금 네가 5개 → 19개로 늘린 후 체감이 확 커졌다면, "**19개를 병렬로 돌리니까 괜찮겠지**"가 실제로는 안 통하는 상황일 가능성이 있다.

특히 CPU가 8코어/16스레드 같은 환경이면 19개 병렬화 자체는 거의 포화되고, 동시에 각 sklearn GPR가 내부적으로 BLAS를 쓰면서 **CPU oversubscription**이 발생할 수도 있다.

즉:

```
19 GPR
x
joblib process/thread
x
BLAS threads
```

가 겹치면 생각보다 효율이 안 나올 수 있다.

이것도 확인해야 한다.

결론

나는 순서를 이렇게 잡을 거야.

가장 먼저

```
19개 GPR 유지
↓
19개 각각 hyperparameter optimization ❌
↓
kernel optimization 횟수 대폭 감소
↓
나머지는 optimizer=None으로 fit
```

그리고 lane 14개는 **kernel sharing**까지 적용.

그러면 **GPR 개수를 19 → 5개로 줄이지 않으면서도, 정확히 19개의 예측값은 그대로 얻을 수 있어.**

즉 네가 원하는 건 사실

"19개의 출력을 유지하면서 19번의 비싼 GPR 최적화를 없애는 것"

이고, 현재 코드 구조에서는 이게 충분히 가능해.

원본 `ML.py` 기준으로라면 `_fit_gpr()` 와 `_fit_models()` 두 군데만 크게 손보면 되는 방향이야. [GitHub](#)



Cloudflare

광고

Deploy serverless functions
Start building for free.

너가 하는 소리가 잘 이해가안감

맞아. 내가 너무 복잡하게 설명했어. 핵심만 아주 단순하게 말할게.

지금 네 상황은 이거야.

기존

GPR 5개 학습 → 빠름

변경 후

GPR 19개 학습 → 확 느려짐

그래서 네가 원하는 건 **19개 출력은 그대로 유지하면서 학습시간만 줄이는 것이지.**

그렇다면 내가 말한 방법 중에서 일단 다른 건 잊어도 되고, 딱 이것만 보면 돼.

지금 코드가 하는 일

GPR 하나를 학습할 때 단순히 데이터만 넣는 게 아니라,

데이터 넣기

↓

"이 GPR의 kernel 파라미터를 뭐로 할까?"

↓

최적값 찾기

↓

최적화된 GPR 완성

을 한다.

그리고 현재는 이걸 **19개 모델 각각에 대해** 한다.

즉 대략:

GPR 1 → 최적화 → 학습

GPR 2 → 최적화 → 학습

GPR 3 → 최적화 → 학습

...

GPR 19 → 최적화 → 학습

그래서 5개에서 19개로 늘리면 당연히 시간이 많이 늘어난다.

내가 말한 "kernel 공유"가 무슨 뜻이냐면

GPR 19개 자체를 5개로 줄이는 게 아니다.

19개는 그대로 둔다.

다만 이런 식으로 하는 거야.

GPR 1 → kernel 최적화 → 학습

GPR 2 → kernel 최적화 안 함 → 학습

GPR 3 → kernel 최적화 안 함 → 학습

...

즉,

비싼 "최적화"는 일부만 하고, 나머지는 이미 정해진 kernel을 사용해서 학습하는 것이야.

그래서

현재

19개 × 비싼 최적화

를

개선

몇 개 × 비싼 최적화

+

나머지 × 싼 학습

으로 바꾸자는 이야기야.

그런데 여기서 중요한 점

내가 지금 바로 코드를 그렇게 바꾸자고 하는 건 아니야.

왜냐하면 먼저 네 코드에서 **19개 GPR 학습시간 중에서 진짜로 optimizer**가 얼마나 차지하는지 확인해야 해.

예를 들어 실제로:

```
GPR fit 전체      20초
그중 kernel 최적화 18초
```

라면 내가 말한 방법이 엄청 효과적이고,

반대로

```
GPR fit 전체      20초
kernel 최적화      2초
```

라면 다른 방법을 찾아야 해.

그래서 제일 먼저 할 일

네 `ML.py` 에 시간 측정을 조금 넣어서 딱 이렇게 확인하는 게 좋다.

```
19개 GPR 전체 학습:  20.3 sec
```

그리고 가능하면 각 모델별로:

```
obj1 : 0.8 sec
obj2 : 0.7 sec
obj3 : 1.2 sec
...
```

이렇게.

그러면 **"19개라서 단순히 19배 가까이 느려진 건지"**, 아니면 특정 **GPR** 몇 개가 유독 오래 걸리는지 바로 알 수 있어.

그리고 한 가지 정정할게.

내가 아까 **"19개 중 일부만 hyperparameter optimization 하면 된다"**고 꽤 적극적으로 말했는데, 이건 네 모델들의 출력 특성이 정말 비슷해서 kernel을 공유해도 되는지 확인하고 해야 한다. 무작정 적용하면 예측 성능이 나빠질 수 있어.

그래서 지금 단계에서는 **kernel** 공유부터 바로 적용하는 것보다, 먼저 **19개 GPR** 중 정확히 어디에서 시간이 쓰이는지 측정하는 게 맞아.

원하면 내가 네 `ML.py` 에서 딱 몇 줄만 추가해서 **GPR 19개의 학습시간을 측정하는 코드**를 정확히 짚어줄게.

그렇게 코드를 바꿔서 쥘볼수있어?

응. 이번에는 네 코드 구조를 그대로 유지하면서, 먼저 “19개 GPR 각각 얼마나 걸리는지”를 찍도록 바꾼 버전을 주는 게 가장 안전해.

그리고 한 가지 중요한 걸 확인했어. 현재 `ML.py` 에는 이미 **5회마다 kernel 재최적화**하는 구조가 들어가 있어. `refresh=True` 인 회차에는 19개 모두 optimizer를 다시 돌리고, 나머지 회차에는 저장된 kernel을 재사용한다.  [GitHub](#)

그래서 아래 수정은 알고리즘을 바꾸지 않는다. 단지 실제 병목을 정확히 측정한다.

바꿀 부분

맨 위 import에 `time` 을 추가하고:

Python

실행됨

```
import time
```

그리고 `_fit_models()` 를 아래처럼 바꿔.

Python

실행됨

```
def _fit_models(df, use_cache=False):
    """목적함수 + 제약조건별 GPR을 따로 학습.

    추가 기능:
    - 각 GPR의 실제 학습 시간을 출력
    - 전체 GPR 학습 시간 출력
    - refresh 여부와 재사용 kernel 여부 표시
    """
    global _ROUNDS_SINCE_REFRESH

    Xs = _normalize(df[PARAM_NAMES].values)

    refresh = (
        (not use_cache)
        or (not _KERNEL_CACHE)
        or (_ROUNDS_SINCE_REFRESH >= _REFRESH_EVERY)
    )

    if use_cache:
        _ROUNDS_SINCE_REFRESH = 0 if refresh else _ROUNDS_SINCE_REFRESH + 1

    print("\n" + "=" * 70)
    print(
```



```

        f"[GPR 학습 시작] "
        f"데이터={len(df)}점 | "
        f"모델={len(MODELED)}개 | "
        f"refresh={refresh}"
    )

    if refresh:
        print(" → 이번 회차는 hyperparameter 재최적화")
    else:
        print(" → 이전 kernel 재사용 (optimizer=None)")

    def _one(name, use_log):
        start = time.perf_counter()

        y = df[name].values.astype(float)
        y = np.log(y) if use_log else y

        warm = None if refresh else _KERNEL_CACHE.get(name)

        gpr = _fit_gpr(
            Xs,
            y,
            warm_kernel=warm
        )

        elapsed = time.perf_counter() - start

        mode = "OPT" if warm is None else "WARM"

        print(
            f" [{mode:4s}] "
            f"{name:<25s} "
            f"{elapsed:8.3f} sec"
        )

        return name, gpr, use_log, elapsed

    total_start = time.perf_counter()

    fitted = Parallel(
        n_jobs=-1
    )(
        delayed(_one)(name, use_log)
        for name, use_log in MODELED
    )

    total_elapsed = time.perf_counter() - total_start

    models = {}

    for name, gpr, use_log, elapsed in fitted:
        models[name] = (gpr, use_log)

```

```

        if use_cache and refresh:
            _KERNEL_CACHE[name] = gpr.kernel_

print("-" * 70)
print(f"[GPR 학습 완료] 전체: {total_elapsed:.3f} sec")

# 모델별 순수 학습시간 합계
model_sum = sum(x[3] for x in fitted)

print(f"[19개 학습시간 합계] {model_sum:.3f} sec")
print(
    f"[병렬화 효과] "
    f"순차합계 / 실제시간 = "
    f"{model_sum / max(total_elapsed, 1e-9):.2f}x"
)

slowest = sorted(
    fitted,
    key=lambda x: x[3],
    reverse=True
)[:5]

print("\n[가장 오래 걸린 GPR TOP 5]")
for name, _, _, elapsed in slowest:
    print(f" {name:<25s} {elapsed:8.3f} sec")

print("=" * 70 + "\n")

return models

```

이렇게만 바꾸면 돼.

실행하면 이런 식으로 나올 거야

예를 들어:

```

=====
[GPR 학습 시작] 데이터=85점 | 모델=19개 | refresh=True
→ 이번 회차는 hyperparameter 재최적화

```

```

[OPT ] pressure_drop           1.823 sec
[OPT ] temp_std                0.912 sec
[OPT ] max_temp                1.104 sec
[OPT ] pass1_lane1             5.932 sec
[OPT ] pass1_lane2             5.641 sec
[OPT ] pass1_lane3             5.812 sec
...
[OPT ] pass2_lane7             5.731 sec
[OPT ] power_module_flow       0.845 sec

```

[OPT] weight 0.402 sec

[GPR 학습 완료] 전체: 38.421 sec
[19개 학습시간 합계] 94.217 sec
[병렬화 효과] 순차합계 / 실제시간 = 2.45x

[가장 오래 걸린 GPR TOP 5]
pass1_lane1 5.932 sec
pass2_lane4 5.881 sec
pass1_lane3 5.812 sec
pass2_lane7 5.731 sec
pass1_lane2 5.641 sec
=====

이 결과가 굉장히 중요해.

왜냐하면 네가 지금 원하는 건 결국:

"19개로 늘어난 게 문제인데, 19개 모델 중 어떤 부분을 줄이면 되느냐?"

이거잖아.

이걸 한번 돌리면 바로 알 수 있어.

예를 들어 결과가:

```
pressure_drop      0.5초
temp_std           0.4초
max_temp           0.5초

lane 14개          각각 5~6초

constraint         0.3초
```

라면 범인은 거의 확실하게 lane GPR들이야.

그러면 그다음에는 내가 말한 lane 쪽만 최적화하면 된다.

반대로 19개가 전부:

```
0.5초
0.6초
0.4초
...
```

라면 모델 하나하나의 문제가 아니라 19개 병렬 처리 자체/BLAS/프로세스 오버헤드를 봐야 해.

그리고 이것도 확인 가능해

현재 코드에서는 `refresh=True` 이면 실제로 kernel optimizer를 돌리고, `refresh=False` 이면 `optimizer=None` 으로 바로 fit한다. [GitHub](#)

따라서 연속으로 몇 회 돌려보면:

```
회차 1
refresh=True
→ 40초

회차 2
refresh=False
→ 5초

회차 3
refresh=False
→ 5초

회차 4
refresh=False
→ 5초

회차 5
refresh=False
→ 5초

회차 6
refresh=True
→ 40초
```

같은 패턴이 보일 수도 있어.

그렇다면 "19개를 줄이는 것"보다 "19개 중 몇 개나 hyperparameter optimization을 해야 하는가"가 핵심 문제라는 게 확실해진다.

현재 코드의 `_REFRESH_EVERY = 5` 와 kernel cache 구조가 바로 이 동작을 결정하고 있어.

[GitHub](#)

내가 추천하는 건

지금은 19개 GPR 자체를 줄이지 말고, 위 코드로 한 번 돌려봐.

그 결과를 나한테 그대로 붙여주면 내가:

```
19개 중 어떤 GPR이 병목인지
→ optimizer가 문제인지
```

- lane 모델만 문제인지
- 병렬화가 문제인지
- 어떤 부분을 바꾸면 되는지

를 네 실제 숫자 기준으로 바로 잡아줄게.

AI 에이전트 구조(아키텍처) 기술 개요 자료가 필요하면 이 쪽.



MuleSoft

광고

Improve AI Agent Architecture

Get the technical overview for MuleSoft Agent Fabric.

전체 코드를 달래니까

24s 동안 처리함

맞아. 내가 아까 부분 코드만 줘서 답답했겠다.

원본 `ML.py` 전체를 그대로 가져와서, 다른 알고리즘은 건드리지 않고 **19개 GPR 각각의 학습시간 / 전체 학습시간 / 가장 느린 GPR TOP 5**가 출력되도록 수정한 전체 파일을 만들었어. 원본 브랜치의 현재 `ML.py` 를 기준으로 작업했고, 기존의 19개 모델·병렬화·kernel cache·5회 refresh 구조는 그대로 유지했어. [GitHub +1](#)

수정된 전체 ML.py 다운로드

실행하면 대략 이렇게 확인할 수 있어:

```
=====
[GPR 학습 시작] 데이터=83점 | 모델=19개 | refresh=True | mode=재최적화
[OPT] pressure_drop          1.234 sec
[OPT] temp_std               0.812 sec
[OPT] max_temp               1.105 sec
[OPT] pass1_lane1           5.721 sec
...
-----
[GPR 학습 완료] 실제 경과시간 : 12.431 sec
[19개 모델 시간 합계]       : 71.284 sec
[병렬화 배율]               : 5.74x
[느린 GPR TOP 5]
pass1_lane1                 5.721 sec
pass2_lane3                 5.603 sec
```

이걸 실제 네 환경에서 한 번 돌려서 출력 결과를 그대로 보내주면, 그 숫자를 기준으로 다음 단계에서 **19개**를 유지하면서 실제 학습시간을 줄이는 수정본을 만들자.

특히 현재 코드상 GPR 19개는 joblib 병렬 학습이고, 5회마다 hyperparameter를 다시 최적화하도록 되어 있어서, 어느 쪽이 병목인지 먼저 보는 게 핵심이야.  [GitHub](#)

ML_timing.py
코드

근데 현재 5회기준으로 더 빠르고 더 느리고 그런거있긴하지?

응. 현재 코드에는 5회 기준으로 느린 회차와 빠른 회차가 확실히 있어.

정확히는 `_REFRESH_EVERY = 5` 때문에 `refresh=True` 가 되는 회차에서 **GPR의 hyperparameter**를 다시 최적화하고, 그 외 회차에서는 이전에 저장한 kernel을 재사용해.

대략 이렇게 동작해:

```
1회 → 재최적화 → 느림
2회 → kernel 재사용 → 빠름
3회 → kernel 재사용 → 빠름
4회 → kernel 재사용 → 빠름
5회 → kernel 재사용 → 빠름
6회 → 재최적화 → 느림
7회 → kernel 재사용 → 빠름
...
```

다만 정확한 기준은 `_ROUNDS_SINCE_REFRESH` 가 어떻게 증가하느냐라서, 코드의 카운터 기준으로는 "정확히 1, 6, 11..."처럼 보일 가능성이 높아.

즉 네가 체감하기에

"어떤 CFD 결과 넣었을 때는 GPR이 유난히 오래 걸리고, 어떤 때는 빨리 끝난다"

면 이 5회 refresh가 원인일 가능성이 꽤 높아.

그리고 더 중요한 건, 19개로 늘렸을 때 이 차이가 훨씬 커졌을 가능성이 있다는 거야.

예를 들어 기존 5개일 때:

일반 회차 2초
refresh 회차 8초

였다면 19개로 늘린 뒤에는:

일반 회차 5초
refresh 회차 30초

같은 식으로 체감 차이가 커질 수 있어.

그래서 지금 우리가 확인해야 할 건 딱 하나야:

실제로 `refresh=True` 회차와 `refresh=False` 회차의 시간이 얼마나 차이 나는지.

내가 방금 준 `ML_timing.py` 에는 그걸 확인할 수 있게 출력하도록 해놨어.

실행 결과에서 `[GPR 학습 시작] ... refresh=True/False` 부분하고 전체 시간이 나오면 바로 판단할 수 있어.

저 코드가 기존코드 토시하나 안건드리고 그냥 시간만 체크할 수있게 넣은거지?

응. 의도는 정확히 그거야.

내가 준 `ML_timing.py` 는 기존 GPR 로직을 바꾸지 않고, `_fit_models()` 안에 시간 측정용 `time.perf_counter()` 와 출력문만 추가한 버전이야.

즉 그대로 유지한 것:

- GPR 19개
- `Parallel(n_jobs=-1)`
- `refresh` 5회 주기
- `_KERNEL_CACHE`
- `warm_kernel`
- GPR kernel / hyperparameter 설정
- candidate 65,536개
- IMSE 2,000개
- 전체 adaptive sampling 로직

추가한 것은:

- 각 GPR 하나가 몇 초 걸렸는지

- 19개 전체가 실제로 몇 초 걸렸는지
- 19개 시간을 단순 합산하면 얼마인지
- 가장 오래 걸린 GPR TOP 5
- 해당 회차가 refresh=True 인지 False 인지

이것만이야.

다만 "토씨 하나도 안 건드렸다"라고 100% 표현하려면 실제 파일 diff를 한번 확인하는 게 가장 정확해. 내가 만든 파일과 원본을 비교해서 정말 측정 코드만 추가됐는지 확인해줄 수도 있어.



Cloudflare

광고

Deploy serverless functions
Start building for free.

코드 딱 디버깅했을때 뜨는 건데 별로안걸리는 거 아닌가

[GPR 학습 완료] 실제 경과시간 : 8.435 sec

[19개 모델 시간 합계] : 11.219 sec

[병렬화 배율] : 1.33x

[느린 GPR TOP 5]

lane1_03 0.889 sec

temp_std 0.767 sec

pressure_drop 0.699 sec

lane1_01 0.673 sec

lane2_05 0.667 sec

더 보기

응. 저 결과만 보면 GPR 학습 자체가 엄청 오래 걸리는 건 아니야.

현재 측정값은:

19개 순차 시간 합계 = 11.219초

실제 병렬 실행 시간 = 8.435초

즉 네 환경에서 19개 GPR 전체 학습이 약 8.4초 정도 걸린다는 뜻이야.

그리고 제일 오래 걸린 것도:


```
lane1_03      0.889초
temp_std      0.767초
pressure      0.699초
```

이러서 특정 **GPR** 하나가 미친 듯이 느린 상황도 아니고.

그러면 중요한 결론

네가 체감상

"해석 끝나고 GPR 때문에 한참 기다린다"

라고 느끼는 거라면, 이 **8.4초**만으로는 설명이 안 될 가능성이 높아.

특히 현재 코드에는 GPR fitting 외에도:

```
GPR fit
↓
candidate 65,536개 예측
↓
IMSE 계산
↓
후보 선택
↓
다음 점 계산
```

이 과정이 있기 때문에, 네가 기다리는 전체 시간 중 **GPR fit**은 일부일 가능성이 높아.

그리고 한 가지 더 중요해.

지금 출력에 `refresh=True/False` 가 안 보이는 상황이면, 이 **8.4초**가 **hyperparameter**를 다시 최적화하는 회차인지 일반 회차인지를 아직 구분해야 해.

지금 바로 확인하면 좋은 것

GPR 학습 시작 부분에 이런 게 있었을 거야.

```
refresh=True
```

인지

```
refresh=False
```

인지 봐줘.

만약 `refresh=True` 인데도 **8.4초**밖에 안 걸린다면:

19개로 늘린 것 자체가 현재 체감시간의 주범은 아닐 가능성이 매우 높다.

반대로:

```
refresh=False → 3~5초  
refresh=True → 30초
```

같은 결과가 나오면 이야기가 완전히 달라져.

지금 네 결과만 보면 나는 일단 **GPR 19개 fitting**을 더 줄이는 작업은 안 할 것 같아. 먼저 전체 `get_next_params()`가 몇 초 걸리는지 쪼개서 봐야 해.

아니 지금은 딱 디버깅했을때 해석하기 전에 걸리는 시간인거고
문제는 해석 이후에 다음해석으로 넘어가기 전 과정에서 시간을 너무잡아먹는단얘기임 그니가 지금 해석이 돌아가고있는
끝난후에 작업되는것도 시간기록남음?

아, 이제 정확히 이해했어. 네가 측정한 8.4초는 첫 해석 전에 `get_next_params()`가 실행될 때의 **GPR 시간**이고, 진짜 문제는:

```
해석 시작  
↓  
CFD/해석 수행  
↓  
해석 완료  
↓  
update_ml()  
↓  
GPR/예측/IMSE/다음점 탐색  
↓  
다음 해석 시작
```

여기서 **해석 완료** → **다음 해석 시작** 사이가 너무 길다는 거지.

그렇다면 내가 앞에서 준 `ML_timing.py` 만으로는 부족해.

현재 넣어둔 시간 측정은 `_fit_models()` 내부만 재기 때문에, 해석이 끝난 뒤 실행되는 전체 과정 중:

- `update_ml()`
- `_fit_models()`

- `_gpr_suggest()`
- 65,536 candidate prediction
- IMSE 계산
- 다음 후보 선택
- 기타 CSV 저장/데이터 처리

가 각각 얼마나 걸리는지는 전부 기록되지 않아.

그래서 지금 필요한 건

해석 후부터 다음 해석 직전까지를 통째로 계측해야 해.

예를 들어 출력이 이렇게 나와야 정확해:

```
=====
[POST-SOLVE TIMING]

CFD 결과 읽기           :   0.12 sec
update_ml()            :   0.08 sec

GPR 전체 학습           :   8.43 sec
GPR prediction          :   2.17 sec
IMSE 계산               :  24.81 sec
다음 후보 선택           :   0.31 sec
CSV 저장                :   0.05 sec

-----
해석 완료 → 다음 해석 시작 : 35.97 sec
=====
```

그러면 바로 알 수 있어.

예를 들어 실제로:

```
GPR fit    8 sec
IMSE       30 sec
```

라면 **GPR 19개를 아무리 최적화해도** 체감시간은 거의 안 줄어들텐데, IMSE를 봐야 하는 거고,

반대로:

```
GPR fit    35 sec
IMSE        5 sec
```

이면 그때 19개 GPR 학습을 줄이는 게 맞아.

그리고 네가 지금 궁금한 부분에 대한 답

아니. 지금 받은 `ML_timing.py`에는 해석이 끝난 뒤부터 다음 해석까지의 전체 시간은 기록되지 않아.

지금은 `_fit_models()` 시간만 제대로 측정한 상태야.

그래서 내가 다음에는 원본 `ML.py` 전체를 기준으로 CFD가 끝난 뒤 호출되는 `update_ml()` 부터 다음 `get_next_params()` 가 끝날 때까지 전부 타이머를 심은 버전을 만들어야 해.

그렇게 해야 우리가 "19개 GPR 때문에 느린 건지, 아니면 GPR 이후 IMSE/후보탐색 때문에 느린 건지" 확실하게 잡을 수 있어.

그럼 그 시간을 보려면 코드어케바꿔야됨 ML.PY 코드만 바꾸면됨?

응. 일단 `ML.py` 만 바꿔서 확인할 수 있어.

다만 조건이 하나 있어: 해석 프로그램이 `ML.py` 의 `update_ml()` 과 `get_next_params()` 를 호출하는 현재 구조라면 `ML.py` 만 수정하면 된다.

네가 알고 싶은 시간을 이렇게 나누면 돼.

```
해석 종료
↓
update_ml()
├─ 해석 결과 읽기
├─ 데이터 추가/저장
└─ 기타 처리
↓
get_next_params()
├─ GPR 학습
├─ 후보 65,536개 예측
├─ IMSE 계산
└─ 다음 해석점 결정
↓
다음 해석 시작
```

그래서 `ML.py` 에 타이머를 넣어서 최소한 이렇게 찍히게 만들면 된다.

```
=====
[POST CFD -> NEXT CFD TIMING]

update_ml 전체      : 0.12 sec
GPR fitting         : 8.43 sec
GPR prediction      : 2.15 sec
```

```
IMSE                : 24.73 sec
candidate selection : 0.18 sec
기타                : 0.31 sec
```

```
-----
전체                : 35.92 sec
=====
```

그러면 해석이 끝난 후 36초를 기다리는데 그중 25초가 IMSE인지, 8초가 GPR인지 바로 알 수 있어.

다만 내가 지금 바로 전체 코드를 수정하려면 내가 실행 중인 최신 `ML.py` 파일 기준으로 작업하는 게 정확해. 지금 대화에서 내가 본 GitHub 주소의 코드와 네 로컬에서 실제 실행하는 코드가 완전히 같은 버전인지 확인해야 하거든.

현재 실행 중인 `ML.py` 를 이 채팅에 첨부해줘. 그러면 그 파일에서 해석 후 구간 전체를 계측해서, 기존 계산은 하나도 건드리지 않고 시간 측정만 추가한 완성본으로 줄게.

너가 준 이코드잖아

```
"""
V4 유로 8변수 - GPR 대체모델 + IMSE 적응 샘플링 + 예측오차
V3 (Code2/ML.py) 대비 변경점 - 두 가지. 둘 다 V3 데이터 (Code2/ML.py)
    ① 학습 대상: vel_cv_pass1/pass2 (2개) → 레인별 유량
        CV = std(레인유량) / mean(레인유량) 인데, 좋은 설계일수록
        같아져서 분자가 "거의 같은 수들의 차이"가 된다. 그래서
        CV에서는 크게 증폭된다 - 레인 유량을 1% 오차로 맞춰도
        진짜 CV 2% → CV 오차 16.7% / CV 5% → 6.0%
        즉 CV를 1%로 맞히려면 레인 유량을 0.1%까지 맞춰야 하
```

더 보기

35s 동안 처리함

맞아. 내가 방금 붙여준 그 코드가 내가 앞에서 준 코드고, 그 코드라면 `ML.py` 하나만 바꿔서 네가 원하는 시간을 볼 수 있어.

이번에는 정확히 해석이 끝난 뒤 → 다음 해석점이 결정될 때까지를 계측하도록 만들었어.

수정된 전체 `ML.py` 다운로드

기존 계산 로직은 유지하고, 시간 측정만 추가했어. 특히 다음을 따로 짭니다:

```

↓
update_ml()
↓
GPR fitting
↓
65,536개 후보 sigma prediction
↓
IMSE
↓
다음점 19개 prediction
↓
get_next_params() 종료

```

[illegible]

GPR fitting	8초
sigma prediction	3초

라면, **19개 GPR**을 줄이는 것보다 **IMSE**가 진짜 병목인 거고,

GPR fitting	35초
IMSE	3초

라면 그때 **19개 GPR 최적화** 쪽을 줄여야 해.

그리고 지금 네가 보여준 기존 결과의 `8.435 sec` 는 **GPR fitting**만의 시간이었고, 이 새 버전에서는 그 뒤의 **65k prediction**과 **IMSE**까지 전부 따로 측정돼.

한 번 적응샘플링이 진행된 후 출력된 `[다음 해석점 결정 구간 TIMING]` 부분을 보여주면, 그 숫자로 바로 어느 부분을 손대야 할지 판단할 수 있어.

ML_post_timing.py
코드

병렬화 배율은 먼뜻임

병렬화 배율 은 **19개 GPR**을 하나씩 순서대로 돌렸을 때 걸리는 시간과, 실제로 동시에 돌렸을 때 걸린 시간의 비율이야.

네 결과가:

```
[19개 모델 시간 합계] : 11.219 sec
[GPR 학습 완료] 실제 경과시간 : 8.435 sec
[병렬화 배율] : 1.33x
```

이면,

$$11.219 \div 8.435 \approx 1.33$$

이라는 뜻이야.

즉,

순차적으로 19개를 돌렸으면 약 11.2초
→ 현재 병렬로 돌리니까 실제로는 8.4초

정도 걸렸다는 의미야.

그런데 1.33배밖에 안 나오는 게 중요해

19개를 병렬로 돌리는데 1.33배밖에 안 빨라졌다는 건 **CPU 코어를 19개가 완전히 활용해서 엄청 빨라진 상황은 아니라는 뜻이야.**

예를 들어 이상적으로 8코어를 아주 잘 활용하면 훨씬 큰 배율이 나올 수도 있는데, 실제로는:

```
GPR 하나당 작업량
+
joblib 프로세스/스레드 오버헤드
+
CPU 코어 수
+
메모리 대역폭
+
sklearn/BLAS 내부 병렬화
```

때문에 그렇게 안 나와.

그래서 네 결과만 놓고 보면:

19개 GPR 자체의 계산량은 합쳐서 11초 정도이고, 현재 병렬화로 8.4초까지 줄인 상태

라고 보면 돼.

그리고 중요한 건 **8.4초가 네가 실제로 기다리는 전체 시간이 아니라는 것**이야.

우리가 방금 만든 `ML_post_timing.py` 로 보면 그 8.4초 뒤에 **65,536 후보 prediction + IMSE** 가 얼마 걸리는지 확인할 수 있어.

그 결과를 보면 어디를 줄여야 하는지가 거의 바로 나온다.

일단 해석을 1회 진행을 해봐야 되는 거네

응, 맞아. 지금은 한 번 실제로 해석을 끝까지 돌려보는 게 가장 중요해.

현재 네가 확인한 **8.4초** 는 GPR fitting 부분이고, 우리가 진짜 알고 싶은 건:

해석 종료 → `update_ml()` → **GPR** → **65k 후보 탐색** → **IMSE** → 다음 해석점 결정

전체가 실제로 몇 초 걸리는지야.

그래서 `ML_post_timing.py` 로 적응샘플링이 실제로 한 번 실행되는 상황까지 가야 의미 있는 로그가 나와.

그리고 첫 80개 DOE 구간이라면 GPR 적응샘플링이 아직 안 들어갈 수 있으니, `N_DOE` 이후의 해석, 즉 `_gpr_suggest()` 가 호출되는 회차의 로그가 필요해.

그때 아래 부분만 가져오면 돼:

```
[POST-CFD -> NEXT-CFD 전체 시간]
```

특히

```
GPR fitting
후보 생성 + 거리필터
1단계 sigma prediction
2단계 IMSE
get_next_params 전체
```

이 숫자를 보면 정확히 어디를 줄여야 하는지 판단할 수 있어.



Cloudflare

광고

Deploy serverless functions
Start building for free.

아니 그럼 GPR때매 오래걸리는건 아닌거아냐? 뭐지 도대체 뭐가문제야

맞아. 지금까지 나온 숫자만 보면 "GPR 19개 fitting 자체가 엄청 느려서 문제"라고 보기는 어려워.

네가 측정한 건:

19개 GPR 순차시간 합계	11.2초
실제 병렬시간	8.4초

이 정도면 8초 자체가 전체 대기시간을 설명하기엔 작을 가능성이 높아.

다만 아직 범인을 확정할 수는 없어. 네 코드에서 해석이 끝난 뒤에는 GPR fitting 말고도 꽤 무거운 게 있어.

해석 종료

↓

```

update_ml()
↓
GPR 19개 fit ← 약 8.4초 확인
↓
65,536개 후보 생성/거리 계산
↓
17개 GPR의 65k개  $\sigma$  prediction
↓
상위 2,000개 선정
↓
17개 GPR × 2,000개 × 1,024 reference
→ IMSE 계산 ← 여기 강력 의심
↓
최종 후보 선택
↓
다음 해석

```

특히 네 코드의 IMSE는 단순한 숫자 몇 개 계산이 아니야. `_imse_reduction()` 에서 각 GPR마다 `2000 × 1024` 규모의 공분산 계산을 하고, 이것 **active objective** 여러 개에 대해 반복한다.

그래서 지금 단계에서 내가 가장 의심하는 건:

① IMSE 계산 → ② 65k 후보에 대한 prediction → ③ 그 다음 GPR fitting

순서야.

그리고 중요한 게 하나 더 있어. 네가 처음 말한 "해석이 끝난 후 다음 해석으로 넘어가기 전에 한참 멈춘다"가 수분 단위라면, 8초짜리 GPR fitting만으로는 절대 설명이 안 될 수 있어.

그래서 이제는 추측하면 안 되고, 방금 만든 `ML_post_timing.py` 로 실제 적응샘플링 한 회차를 측정하는 게 맞아.

그 결과가 예를 들어:

```

GPR fitting      8 sec
sigma prediction  4 sec
IMSE              90 sec

```

이면 범인은 바로 IMSE고,

```

GPR fitting      8 sec
sigma prediction  4 sec
IMSE              5 sec
전체             20 sec

```

이면 `ML.py` 밖의 해석 후처리나 호출부를 봐야 해.

그러니까 지금은 "GPR이 문제 아니다"라고 완전히 결론 내리는 단계가 아니라, GPR fitting은 일단 주범 후보에서 한 단계 내려놓고 `ML_post_timing.py` 로 전체 구간을 찍어보는 단계라고 보면 돼.

그리고 만약 그 결과 IMSE까지 합쳐도 몇십 초밖에 안 나오는데 실제로는 몇 분씩 기다린다면, 그때는 거의 확실하게 `ML.py` 를 호출하는 해석 코드 쪽을 봐야 해.

GPR × 2,000개 × 1,024 reference 이걸 왜이렇게함? 이해가안됨

좋아. 이 부분이 현재 코드에서 제일 헷갈릴 만한 부분이야. 왜 **2,000 × 1,024**를 하느냐부터 아주 쉽게 설명할게.

네 코드의 목적은 단순히:

"다음에 어디가 예측이 가장 불확실하지?"

만 찾는 게 아니야.

지금은:

"어디에 CFD 점 하나를 추가하면, 설계공간 전체의 불확실성이 가장 많이 줄어들까?"

를 찾으려고 IMSE 를 쓰고 있어.

그래서 두 종류의 점이 필요해

1. 2,000개 = "다음 실험 후보"

설계공간에서 65,536개를 뽑은 다음 먼저 GPR의 σ 가 큰 곳을 골라.

65,536개 후보

↓

σ 가 큰 후보

↓

2,000개

즉 "다음 해석을 할 만한 후보 2,000개"야.

2. 1,024개 = "설계공간 전체를 대표하는 점"

그런데 후보 하나가 좋은지 판단하려면:

이 후보를 실제로 하나 찍었다고 가정했을 때, 설계공간 전체의 불확실성이 얼마나 줄어드나?

를 계산해야 해.

설계공간 전체를 무한히 다 계산할 수는 없으니까 1,024개의 대표점(`ref`)을 만든다.

설계공간 전체

↓

대표점 1,024개

그러면 후보 하나에 대해 이렇게 계산하는 거야

예를 들어 후보 `C1` 이 있다고 하자.

`C1`을 추가한다고 가정

↓

설계공간 대표점 1,024개 각각에 대해
불확실성이 얼마나 줄어드는지 계산

↓

1,024개 결과의 평균

↓

`C1`의 IMSE 점수

그리고 이걸 후보 2,000개 각각에 대해 한다.

후보 1 → 1,024개 계산

후보 2 → 1,024개 계산

후보 3 → 1,024개 계산

...

후보 2000 → 1,024개 계산

그래서 내가 $2,000 \times 1,024$ 라고 한 거야.

그리고 이걸 각 GPR마다 계산하니까:

GPR 1 → $2,000 \times 1,024$

GPR 2 → $2,000 \times 1,024$

...

GPR 17 → $2,000 \times 1,024$

이런 구조가 되는 거야.

왜 GPR이 17개냐?

네 `MODELED` 는 19개지만,

17개 objective
+
2개 constraint

이야.

적응샘플링 IMSE에는 objective 17개만 사용하고 constraint 2개는 사용하지 않아.

그래서 실제 IMSE 계산은 대략:

$17 \times 2,000 \times 1,024$

규모가 된다.

물론 코드가 진짜 $17 \times 2,000 \times 1,024$ 개의 독립적인 Python 반복문을 도는 건 아니고, NumPy 행렬연산으로 한꺼번에 계산해. 그래도 처리해야 하는 데이터 크기가 상당히 큰 건 맞아.

왜 굳이 이렇게 복잡하게 하냐?

기존의 단순한 방법은:

σ 가 가장 큰 곳 = 다음 실험점

이거야.

그런데 이 방법은 코너나 경계에 몰릴 수 있어.

현재 네 코드 주석에도 V3에서 실제로 그런 문제가 있었고, 그래서 $\sigma \times \text{sparsity}$ 대신 IMSE로 바꾼 이유가 적혀 있어.

IMSE는:

"이 점을 하나 추가했을 때, 전체 설계공간의 불확실성이 얼마나 줄어드는가?"

를 보니까, 단순히 그 점 하나에서 σ 가 큰 것보다 전체적으로 정보가 많이 퍼지는 위치를 찾을 수 있는 거야.

그런데 네 질문의 핵심은 사실 이거지?

"그럼 이 계산이 너무 무거운 거 아닌가?"

맞아. 바로 그 부분이 내가 지금 의심하는 병목이야.

네가 측정한 GPR fitting이 8.4초인데,

```
17개 GPR
x
2,000 후보
x
1,024 reference
```

계산까지 들어가면 **GPR fitting**보다 **IMSE**가 더 오래 걸릴 가능성이 충분히 있어.

특히 `_imse_reduction()` 안에는 단순히 $2,000 \times 1,024$ 만 있는 게 아니라 공분산 행렬 계산과 `cho_solve` 까지 들어가.

그래서 지금은 **candidate** 수를 줄이자고 바로 결정할 게 아니라, 우리가 만든 `ML_post_timing.py` 로 실제 IMSE가 몇 초인지 먼저 보는 게 맞아.

결과가 예를 들어:

GPR fitting	8 sec
σ prediction	3 sec
IMSE	80 sec

이렇게 나오면 그때는 범인이 거의 확실하게 **IMSE**야.

그럼 내가 다음에는 정확도/탐색 성능을 최대한 유지하면서 **IMSE** 계산 자체를 빠르게 만드는 방법을 제안하면 돼.

 **Profound**

 **Profound**

광고

Turn AI Demand Into Content Wins
See top prompts, gaps, and citation lift fast.

근데 왜 2000개야? 무슨기준이야? 그리고왜 1024개야 그건 또 무슨기준임

좋은 질문이야. 결론부터 말하면 **2,000**과 **1,024**는 물리 법칙으로 정해진 숫자가 아니야. 지금 코드에서 사람이 정한 계산 예산(**parameter**)이야.

2,000은 왜?

코드에 이렇게 되어 있어:

Python

실행됨

```
N_CAND = 65536
N_IMSE_CAND = 2000
```

먼저 65,536개 후보를 만든 다음, GPR의 불확실성 σ 가 큰 순서로 상위 2,000개만 IMSE를 계산하도록 한 거야.

즉 기준은:

"65,536개를 전부 IMSE 계산하면 너무 비싸니까, 일단 σ 가 큰 후보 2,000개만 정밀평가하자."

야.

그런데 중요한 점은 "2,000이어야 한다"는 근거는 현재 코드에 없어.

쉽게 말해:

65,536개 전부 IMSE

→ 너무 비쌘

2,000개만 IMSE

→ 적당히 비싸면서 후보를 충분히 많이 본다고 가정

500개만 IMSE

→ 더 빠름

이 중 2,000은 경험적으로 잡은 계산량 설정값이지, GPR 이론에서 나오는 숫자가 아니야.

그럼 1,024는?

이것도 마찬가지로.

Python

실행됨

```
N_IMSE_REF = 1024
```

이 숫자는 설계공간 전체를 대표해서 IMSE를 적분할 때 사용할 reference point 개수야.

IMSE는 원래 개념적으로:

설계공간 전체에서 예측 분산이 얼마나 줄어드는지 적분

해야 하는데, 8차원 공간을 정확하게 적분하기는 너무 비싸.

그래서 현재 코드는:

Python

실행됨

```
ref = qmc.Sobol(...).random(1024)
```

로 설계공간에 1,024개의 Sobol 점을 뿌려서 전체 공간을 대표하게 한 거야.

즉:

실제 설계공간 = 무한히 많은 점
↓
Sobol 1,024개
↓
"전체 공간을 이 1,024개로 대표하자"

인 거야.

여기서도 1,024가 반드시 맞는 숫자는 아니야.

그럼 숫자에 의미가 전혀 없냐?

아예 아무 의미 없이 정한 건 아니야.

1,024

$1024 = 2^{10}$ 이라 Sobol sequence를 사용할 때 다루기 편한 숫자야.

2,000

후보 65,536개 중에서 너무 적지도 않고, IMSE 계산을 감당할 수 있는 상위 후보 수로 잡은 거야.

하지만 둘 다:

"8차원 문제에서는 반드시 2,000 / 1,024를 써야 한다"

같은 이론적 기준은 없어.

사실 여기서 네가 짚은 게 중요해

현재 코드를 보면:

65,536

↓

2,000

↓

1,024

이 숫자들이 **계산량을 엄청 크게 만드는 주요 설정값**이야.

그리고 네가 아까:

"19개 GPR 때문에 시간이 긴 게 아닌 것 같다"

라고 했잖아.

그렇다면 다음으로 의심할 건 바로 이 부분이 맞아.

특히 IMSE는 대충:

```
17개 objective
× 2,000 candidate
× 1,024 reference
```

규모의 계산을 하니까.

그런데 더 중요한 질문이 있어

"2,000과 1,024를 줄이면 결과가 얼마나 나빠지느냐?"

이걸 테스트해야 해.

예를 들어 같은 데이터에서:

```
N_IMSE_CAND = 2000
N_IMSE_REF = 1024
```

로 뽑은 다음 점과

```
N_IMSE_CAND = 1000
N_IMSE_REF = 512
```

로 뽑은 다음 점이 거의 비슷하다면,

계산량 절반 이하로 줄이고도 탐색 결과는 사실상 유지할 수 있어.

반대로 완전히 다른 점이 자주 나온다면 지금 숫자를 유지해야 하고.

그래서 이 두 숫자는 성능과 정확도 사이의 trade-off를 실험으로 결정하는 값이야.

그리고 한 가지 정정

내가 앞에서 $17 \times 2,000 \times 1,024$ 를 너무 직관적으로 설명했는데, 실제 `_imse_reduction()` 은 단순한 삼중 for-loop가 아니라 NumPy 행렬연산으로 한꺼번에 처리해.

그래서 실제 실행시간이 정확히 저 곱셈에 비례하는 건 아니야.

하지만 계산 규모를 이해하는 숫자로써는 맞아.

지금 단계에서는 2,000과 1,024를 함부로 줄이기보다, 네가 실제로 한 번 돌려서 IMSE 가 몇 초 걸리는지 먼저 보는 게 제일 중요해.

그 숫자가 나오면 그때는 내가 "2,000 → 몇 개, 1,024 → 몇 개까지 줄여도 다음 선택점이 거의 안 바뀌는지"를 코드로 검증하는 방향으로 가는 게 가장 깔끔해.

2000이랑 1024 이거 숫자 바꾸면 시간 영향큼?

응. 영향 꽤 커. 특히 2000 쪽은 영향이 크고, 1024 도 무시할 정도는 아니야.

현재 구조를 아주 단순화하면:

`N_IMSE_CAND = 2000`

↓

IMSE를 계산할 후보 수

`N_IMSE_REF = 1024`

↓

각 후보를 평가할 때 참고할 공간상의 점 수

그래서 IMSE 계산량은 대략 두 숫자의 곱에 비례하는 방향이야.

현재:

$2000 \times 1024 = 2,048,000$

인데,

예를 들어

2000 → 1000
1024 → 1024

로 바꾸면 대략 IMSE 쪽 계산량이 **절반** 수준으로 내려갈 수 있어.

$1000 \times 1024 = 1,024,000$

둘 다 절반으로:

2000 → 1000
1024 → 512

하면

$1000 \times 512 = 512,000$

이라서 현재의 **약 1/4** 규모가 돼.

반대로:

2000 → 4000
1024 → 2048

이면 현재보다 대략 **4배** 규모가 돼.

그런데 중요한 차이가 있어

시간 영향은 2000 을 줄이는 게 더 직접적이야.

2000 은 실제로 IMSE를 계산하는 후보 개수라서, 줄이면 검사하는 후보 자체가 줄어들어.

1024 는 설계공간을 대표하는 **reference point** 수라서, 줄이면 각 후보의 IMSE를 계산하는 정밀도가 낮아지는 거야.

즉:

2000 ↓
→ "검사할 후보를 줄임"

1024 ↓
→ "각 후보 평가의 정밀도를 낮춤"

그리고 네 코드에서는 **2,000**이 무조건 필요한 숫자가 아니야.
특히 우리는 지금까지 실제 IMSE 계산시간을 아직 안 봤잖아.

그래서 내가 보기엔:

일단 현재 2000 / 1024 그대로 한 번 돌려서 IMSE가 실제 몇 초인지 확인 → 그 다음 숫자를 조정
이 순서가 가장 안전해.

예를 들어 IMSE가 실제로 60초라면 `2000 → 1000` 테스트 가치가 엄청 크고, IMSE가 3초밖에 안 되면 굳이 건드릴 이유가 없어.

근데 이기법을 쓰면 확실히 좋기는하지?

응. 네 문제에서는 이 기법(IMSE)을 쓰는 이유가 분명히 있어. 다만 “무조건 항상 더 좋다”까지는 아니고, 지금 네가 겪었던 코너 쏠림 문제를 해결하는 데는 상당히 좋은 선택이야.

쉽게 비교하면:

기존 방식: σ 가 큰 곳 선택

"여기 예측이 제일 불확실하네?"

↓

그 점을 다음 CFD로 선택

문제는 불확실성이 큰 곳이 특정 경계/코너에 계속 몰릴 수 있다는 것이야.

네 코드 주석에도 V3에서 실제로 그런 현상이 관찰됐다고 적혀 있지.

현재 방식: IMSE

"이 점 하나를 추가하면
설계공간 전체의 불확실성이
얼마나 줄어들까?"

↓

전체적으로 효과가 큰 점 선택

그래서 단순히 한 점에서 σ 가 큰 것보다 전체 설계공간을 골고루 학습시키는 방향으로 간다.

특히 네 문제는 **8차원 설계공간 + CFD가 한 번 돌 때 비용이 큼** → 실험점 하나하나의 가치가 중요함.

그래서 IMSE가 잘 맞는 편이야.

네 데이터에서 더 중요한 건 실제 검증 결과야

네 코드 설명에 이미 비교 결과가 적혀 있어:

σ 만	→ cornerness 0.717
$\sigma \times \text{sparsity}$	→ 0.845
IMSE	→ 0.461

그리고 V3에서 실제 적응점의 **96.6%**가 8개 변수 중 5개 이상이 경계 10%에 몰리는 문제가 있었다고 되어 있어.

즉 네 데이터에서는 IMSE를 사용한 결과가 단순 σ 보다 훨씬 중앙/공간 전체를 고려하는 방향으로 개선됐다는 실측 근거가 있어.

그래서 나는 IMSE라는 방법 자체는 유지하는 쪽을 추천해.

다만 중요한 건:

IMSE를 유지하면서 계산을 조금 가볍게 만드는 것

이 가능하다는 거야.

즉 내가 지금 보고 있는 방향은

IMSE 없애기	✗
↓	
IMSE 유지	✓
↓	
2000 / 1024가 정말 필요한지 검증	
↓	
가능하면 계산량만 줄이기	

야.

그리고 2000과 1024를 줄인다고 해서 IMSE라는 방법 자체를 포기하는 게 아니야. 같은 IMSE를 좀 더 거칠게 계산하는 거라서, 실제 선택점이 얼마나 달라지는지만 확인하면 돼.

그래서 지금은 방법론은 건드리지 말고, 먼저 실제 실행에서 IMSE가 몇 초 걸리는지 보는 게 가장 맞아. 그 숫자가 나오면 그 다음 판단이 굉장히 쉬워져.

근데 2000이 쥔 불확실한 상위 2000개 뽑는거지?

응. 정확히는 그래.

코드에서 먼저 65,536개 후보를 만들고:

Python

실행됨

```
sig_sum += sig / (sig.max() + 1e-12)
```

각 후보에 대해 **활성 objective**들의 예측 표준편차 σ 를 합산해.

그다음:

Python

실행됨

```
n_keep = min(N_IMSE_CAND, len(cand_norm))  
top = np.argpartition(-sig_sum, n_keep - 1)[:n_keep]
```

으로 `sig_sum` 이 큰 상위 **2,000개**를 뽑아.

즉 흐름은 정확히:

```
graph TD
    A[65,536개 후보] --> B[↓]
    B --> C[각 후보의 "전체 목적함수 불확실성" 계산]
    C --> D[↓]
    D --> E["σ 합이 큰 순서로"]
    E --> F[↓]
    F --> G[상위 2,000개]
    G --> H[↓]
    H --> I[이 2,000개만 IMSE 정밀평가]
    I --> J[↓]
    J --> K[최종 1개 선택]
```

다만 한 가지 중요한 차이가 있어.

"가장 불확실한 GPR 하나의 상위 2,000개"가 아니라, 현재 수렴하지 않은 여러 목적함수의 σ 를 정규화해서 합친 `sig_sum` 기준 상위 2,000개야.

그래서 예를 들어 pressure가 불확실한 곳과 lane이 불확실한 곳을 같이 고려해서 2,000개를 고르는 거야.

그리고 최종 선택은 이 2,000개 중에서 σ 가 제일 큰 점을 고르는 게 아니고, IMSE가 가장 큰 점을 고른다.

즉:

σ = 1차 후보 압축

IMSE = 최종 선정

이라고 보면 딱 맞아.

아니면 지금 은 1회 해석할때마다 불확실 점을 판단하잖아 아
니면 예를들어봐바 초기실험계획 80점을 가지고 예측모델만
들잖아 여기서 불확실도를 기준으로 해석점을 하나씩 하나가는
거말고 한 5점 ~ 10점을 뽑는거야 그리고 그 10점을 쭉 해석
하고 재학습하고 이런식으로 가는건어떨

응. 그 방식은 충분히 가능하고, 오히려 네 상황에서는 꽤 합리적인 방법이야.

지금은:

초기 DOE 80점
↓
GPR 학습
↓
다음 점 1개 선택
↓
CFD 1회
↓
GPR 다시 학습
↓
다음 점 1개
↓
...

인데, 네가 제안한 건 **batch adaptive sampling**이야.

초기 DOE 80점
↓
GPR 학습
↓
좋은 후보 10개 선정
↓
CFD 10회 연속 수행
↓
10개 결과 추가
↓
GPR 한 번 재학습

↓
다음 10개 선정
↓
...

이건 해석과 **GPR** 사이의 대기시간을 줄이는 데 굉장히 좋은 방법이 될 수 있어.

왜 빨라지나

현재 방식에서는 adaptive point 하나마다:

CFD
→ GPR
→ 65k 후보
→ IMSE
→ 다음점
→ CFD
→ GPR
→ ...

를 반복해.

그런데 10개 batch로 하면:

GPR
→ 10개 선정
→ CFD
→ CFD
→ CFD
→ ...
→ CFD
→ GPR
→ 10개 선정

이렇게 되니까 **GPR + 후보탐색 + IMSE**를 10번 하는 대신 1번만 하면 돼.

네가 지금 측정한 GPR fitting이 약 8.4초라고 했잖아. 후보탐색/IMSE까지 포함하면 이 절약폭은 더 커질 수 있어.

그런데 중요한 문제가 하나 있어

여기서 그냥 현재 방식으로 10개를 연속해서 뽑으면 안 돼.

왜냐하면 10개를 뽑을 당시 GPR은 80점만 알고 있기 때문이야.

예를 들어:

80개 데이터

↓

GPR

↓

A, B, C, D, E, F, G, H, I, J 선택

하면 A를 CFD한 결과를 아직 GPR이 모르잖아.

그래서 B를 고를 때도 **A가 존재하지 않는다고 생각한 옛날 GPR**을 기준으로 뽑게 돼.

그러면 A와 B가 비슷한 곳에 몰릴 가능성이 생겨.

그래서 batch를 뽑을 때는 "서로 겹치지 않게" 해야 해

여기서 방법이 몇 가지 있어.

방법 1 — 한 번에 10개를 고르되 거리 제한을 강하게

첫 번째 후보를 뽑고,

A 선택

그 주변은 후보에서 제거하고,

B 선택

다시 그 주변 제거...

이런 식으로 **10개가 서로 충분히 떨어지도록** 선택하는 방법.

이게 구현하기 제일 쉬워.

방법 2 — q-point / batch acquisition

좀 더 제대로 하려면

"10개를 동시에 추가했을 때 전체 불확실성이 가장 많이 줄어드는 10개 조합"

을 찾는 방법도 있어.

하지만 이건 지금 코드보다 알고리즘이 훨씬 복잡해져.

네 문제에서는 나는 "5개 batch"부터 시작하는 걸 추천해

예를 들어:

초기 DOE = 80

↓

GPR 학습

↓

5개 선택

↓

CFD 5회

↓

5개 데이터 추가

↓

GPR 학습

↓

다음 5개 선택

↓

...

이렇게.

왜 10개보다 5개냐면 모델이 낡는 정도와 계산 절감 사이의 균형 때문이야.

10개를 한꺼번에 뽑으면:

80점 모델로

80 → 90점 구간을 한 번에 예측

하게 되는데, 5개라면:

80 → 85

85 → 90

으로 더 자주 업데이트되니까 적응성이 좋아.

그리고 여기서 네 IMSE와도 잘 맞아

현재 IMSE가:

"다음 점 하나가 들어왔을 때 전체 불확실성을 얼마나 줄이는가"

를 보는 건데,

batch로 가면 **IMSE**를 이용해 첫 번째 점을 고른 후 그 근처를 제거하고 다음 점을 고르는 방식으로 확장할 수 있어.

즉:

```
GPR
↓
IMSE → 1번 후보
↓
그 후보 주변 제외
↓
IMSE → 2번 후보
↓
그 주변 제외
↓
...
↓
10개 후보
```

이렇게 하면 서로 붙어있는 10개를 뽑는 문제를 어느 정도 막을 수 있어.

네 상황에서 장단점을 보면

장점

가장 큰 장점은 **CFD와 ML** 사이의 왕복 횟수가 줄어드는 것이야.

현재:

```
CFD → ML → CFD → ML → CFD → ML
```

batch:

```
ML → CFD → CFD → CFD → CFD → CFD → ML
```

이라서 **GPR/IMSE** 계산 횟수가 크게 줄어들어.

그리고 CFD가 병렬 실행 가능하다면 더 좋다.

단점

가장 큰 단점은 중간 결과를 보고 방향을 수정하지 못한다는 것이야.

예를 들어 5개 중 첫 번째 CFD 결과를 보고:

"아, 이쪽은 별로네."

라고 알게 되어도 나머지 4개는 이미 선정되어 있어.

그래서 batch 크기가 너무 커지면 adaptive sampling의 장점이 약해진다.

그래서 내가 네 코드라면

처음에는:

```
초기 DOE      80
batch size     5
```

로 테스트해볼 것 같아.

그리고 실제로 돌려보고:

```
batch=1  → 정확도/수렴 좋음, 하지만 느림
batch=5   → 비슷한 수렴 + 시간 크게 감소
batch=10  → 시간 더 감소, 하지만 수렴 효율 저하
```

를 비교하는 거지.

특히 **CFD 한 회 시간이 GPR + IMSE보다 압도적으로 긴 경우에는 batch 크기를 키우는 이득이 작아질 수 있어.** 반대로 네가 말한 것처럼 **CFD가 끝난 뒤 ML 단계에서 몇 분씩 기다리는 상황**이라면 batch 방식의 효과가 꽤 클 수 있어.

핵심은

네가 지금 고민하는 건 사실:

"GPR을 더 빠르게 만들까?"

보다

"GPR을 매번 하지 않도록 만들까?"

에 가까워.

그리고 두 번째 방법이 훨씬 큰 개선을 줄 가능성이 있어.

특히 네 코드에서는 GPR뿐 아니라 **65,536 후보 생성 + IMSE**도 매 adaptive cycle마다 실행되니까, batch 5개를 쓰면 이 비용도 대략 1/5 수준의 호출 횟수로 줄일 수 있어.

다만 정확도를 최대한 유지하려면 batch를 어떻게 선정하느냐가 중요해.

원하면 다음 단계로는 현재 네 `ML.py`를 기준으로 `BATCH_SIZE=5`를 넣어서 "5개 후보를 먼저 뽑고 → CFD 5회 → 한꺼번에 재학습"하도록 바꾸는 구조를 구체적으로 설계해보는 게 맞아.

근데 이러면 예측오차종료기준을 판단할때 5회해석마다 판단하게되겠네 .

맞아. 그대로 **batch=5**로 바꾸면 종료판정도 기본적으로 5회 해석이 끝난 뒤에 한 번 판단하게 돼.

현재 `is_done()` 은 `update_ml()` 이 한 번 끝날 때마다 데이터를 보고:

해석 1 → 오차 확인
해석 2 → 오차 확인
해석 3 → 오차 확인
...

하는 구조인데,

batch 방식에서는:

GPR
↓
5개 선정
↓
해석 5개
↓
5개 데이터 한꺼번에 추가
↓
GPR 재학습
↓
예측오차/종료판정

이 되니까 **중간 1~4번째 결과만으로는 최신 GPR의 종료판정을 할 수 없어.**

그리고 네 종료조건이:

모든 그룹이 1% 이하
+
3회 연속 만족

이기 때문에 **batch=5**라면 더 중요한 문제가 생겨.

예를 들어 **batch** 결과가:

5회차 batch → 조건 만족
10회차 batch → 조건 만족
15회차 batch → 조건 만족

이러면 실제로는 **15개 해석을 한 뒤에야 3번 연속 만족을 확인하게 돼.**

그런데 이건 해결할 수 있어

나는 batch를 쓰더라도 종료판정은 매 해석 결과마다 기록하고, **GPR 재학습만 batch 단위로 하는 방식을 추천해.**

즉:

```
80점
↓
GPR
↓
5개 선정
↓
CFD 1 → 결과 기록
CFD 2 → 결과 기록
CFD 3 → 결과 기록
CFD 4 → 결과 기록
CFD 5 → 결과 기록
↓
GPR 재학습
↓
다음 5개 선정
```

여기서 각 CFD 결과에 대해 예측오차는 **batch를 선정할 당시의 GPR 예측값**과 비교해서 저장할 수 있어.

그러면:

```
해석 81 → 오차 기록
해석 82 → 오차 기록
해석 83 → 오차 기록
...
```

은 가능해.

다만 종료판정 자체를 정말 최신 모델 기준으로 하려면 마지막 batch의 모든 결과가 들어온 뒤 GPR을 한 번 학습해야 해.

그래서 종료조건을 어떻게 정의할지가 중요해

두 가지 선택지가 있어.

A. batch 종료판정

5개 해석
→ GPR 학습
→ 종료판정

가장 단순하고 안정적.

B. 개별 해석 결과는 계속 평가하지만, GPR 업데이트는 batch

5개 해석 결과 기록
→ 각 결과의 예측오차 기록
→ 5개 끝나면 GPR 업데이트
→ 종료판정

이 방식이 네가 지금 가지고 있는 3회 연속 만족 로직을 최대한 보존할 수 있어.

그리고 여기서 **batch=5**가 무조건 좋은 건 아니다.

네 종료조건이 상당히 엄격하게 **3회 연속**으로 되어 있기 때문에 batch를 너무 크게 잡으면 종료가 실제보다 늦게 감지될 수 있어.

그래서 네 문제에서는 나는 오히려:

batch size = 3~5

정도를 먼저 검토할 것 같아.

특히 중요한 건 **batch**로 묶을 때 **5개를 어떻게 고르느냐**야. 그냥 IMSE 상위 5개를 뽑으면 서로 비슷한 위치에 몰릴 수 있으니까, 서로 어느 정도 떨어진 5개를 선정해야 해.

아 진짜 머리아파 이 적응샘플링 그리고 종료판정을 내가 너나 클로드한테 물어볼때마다 방식이 너무 방대해서 진짜 힘들어 ..
아니 이걸 예측오차기준으로 가는게 맞는건지 어떤건 또 LOOCV인가 그걸 추천하던데 미쳐버리겠네

그렇지 만해. 지금 네 문제는 방법이 너무 많아서 고르는 것 자체가 일이 돼버린 상태야. 내가 앞에서 여러 선택지를 한꺼번에 던진 것도 오히려 헷갈리게 만든 것 같아.

일단 다 내려놓고, 네 문제에 맞는 기준 하나를 정하자.

내 판단은 이거야:

현재는 예측오차 기반 종료 + IMSE 기반 적응샘플링 을 유지하는 게 가장 합리적이다.
LOOCV로 갈아탈 필요는 없다.

왜 그런지 아주 간단하게 구분하면 돼.

1. 적응샘플링은 "다음 CFD를 어디서 할까?"

이건 IMSE가 담당.

현재까지의 데이터

↓

GPR

↓

어디를 추가하면 전체 모델이 가장 좋아질까?

↓

다음 CFD 위치

2. 종료판정은 "이제 모델을 믿어도 될까?"

이건 지금 네 코드의 실제 예측오차가 담당.

GPR이 예측

↓

실제로 CFD

↓

예측 vs 실제 비교

↓

오차가 충분히 작은가?

이 둘은 서로 다른 질문이야.

LOOCV는 뭐냐?

LOOCV는 이미 있는 데이터에서:

80개 중 1개 빼기

→ 나머지 79개로 학습

→ 뺀 1개 예측

→ 오차 계산

하는 거야.

즉 현재 가지고 있는 데이터만으로 모델이 얼마나 잘 일반화되는지 평가하는 방법이야.

좋은 방법이긴 한데, 네 문제에서는 중요한 한계가 있어.

네가 원하는 건:

"앞으로 새로운 설계점을 CFD로 더 계산해야 하나?"

잖아.

LOOCV는 기존 80개 안에서 모델이 잘 맞는지 보는 거지, 새로운 영역을 잘 예측할 수 있는지 직접 보장하는 건 아니야.

그래서 적응샘플링을 멈출 근거로 LOOCV만 사용하는 건 나는 추천하지 않아.

네 경우에는 실제 CFD 검증점이 이미 생기잖아

이게 굉장히 중요해.

적응샘플링에서:

GPR 예측

→ CFD 실제값

→ 예측오차 계산

이게 되니까 실제로 새로운 점에 대한 일반화 성능을 직접 측정할 수 있어.

이건 굉장히 좋은 종료 기준이야.

그래서 나는 네가 지금 가지고 있는:

1% 이하

+

3회 연속

+

모든 주요 그룹 만족

이라는 구조를 그대로 유지하는 쪽을 추천해.

그럼 결국 지금 네 알고리즘은 이렇게 정리하면 돼

초기

80점 DOE

↓

GPR 학습

적응

IMSE

↓

다음 CFD 점 1개

↓

CFD

↓

GPR이 그 점을 예측했던 값과 실제값 비교

↓

오차 기록

↓

다시 IMSE

↓

다음 점

종료

최근 3개 적응점에서

모든 종료 그룹의 예측오차 $\leq 1\%$

↓

종료

이 정도면 충분히 논리적으로 깔끔해.

오히려 지금 당장 바꾸지 말아야 할 것

나는 지금 네 코드에서

- IMSE를 다른 adaptive method로 교체
- 예측오차 종료를 LOOCV로 교체
- batch 5개/10개로 변경
- GPR을 multi-output으로 변경

이걸 전부 하지 않는 걸 추천해.

왜냐하면 지금은 방법론이 부족한 게 아니라 이미 방법이 너무 많아.

먼저 하나를 제대로 검증해야 해.

그리고 지금 네가 해야 할 건 딱 하나

현재 방식 그대로 실제 **adaptive CFD**를 몇 회 돌려서 데이터를 보는 것.

우리가 만든 timing 버전으로:

```
CFD 종료
↓
GPR fitting
↓
σ 계산
↓
IMSE
↓
다음점 선택
```

각 시간이 얼마나 걸리는지 보고,

동시에:

```
예측오차:
1회  2.3%
2회  1.4%
3회  0.8%
...
```

가 어떻게 내려가는지를 보면 돼.

그걸 보고 나서야

"이 방법이 실제로 잘 수렴하고 있는가?"

를 판단할 수 있어.

내 추천을 한 문장으로 정리하면

지금은 80점 DOE → GPR → IMSE로 1점씩 추가 → 실제 CFD 예측오차 $\leq 1\%$ 가 3회 연속이면 종료를 기준선(baseline)으로 확정하고, 다른 방법은 당분간 버리자.

LOOCV는 나중에 보조 진단용으로 붙이면 돼. 종료 기준을 바꿀 필요는 없어.

그리고 batch도 지금 당장 도입할 필요 없어. 현재 방식이 실제로 몇 초/몇 분 걸리는지와 오차가 실제로 줄어드는지부터 확인하는 게 먼저야.

그냥 1점씩추가하는거로 계속 가? 지금 방식으로?

응. 지금은 그냥 1점씩 추가하는 방식으로 계속 가는 걸 추천해.

네 상황에서는 그게 제일 단순하고, 검증하기도 좋고, 나중에 설명하기도 쉬워.

현재 방식:

```
초기 DOE 80점
↓
GPR 학습
↓
IMSE로 다음 1점 선택
↓
CFD 1회
↓
실제값 vs GPR 예측값 비교
↓
데이터 1점 추가
↓
GPR 업데이트
↓
다음 1점 선택
↓
...
```

종료는:

```
최근 3회 연속
모든 종료 그룹의 예측오차  $\leq 1\%$ 
↓
종료
```

이걸 일단 **baseline**으로 확정하자.

지금 당장 batch 5/10이나 LOOCV로 바꾸면, 나중에 결과가 좋아졌거나 나빠졌을 때 무엇 때문에 그런지 추적하기가 더 어려워져.

그리고 네가 걱정했던 속도 문제도 아직 **GPR** 자체가 범인인지 **IMSE**가 범인인지 확인 중이니까, 먼저 현재 방식을 한두 회 실제로 돌려서 timing을 보는 게 맞아.

내가 지금이라면 알고리즘은 더 안 건드리고, 실행시간 계측 → 실제 오차 추이 확인 → 그다음 최적화 순서로 갈 거야.

근데 이게 넘 오래걸린다고 $\pi\pi$

아 ㅋㅋ 이제 핵심이 완전히 이해됐어. 방법론은 지금 방식으로 가고 싶은데, 문제는 CFD 한 번 끝날 때마다 다음 점을 고르는 계산이 너무 오래 걸리는 거지. 😭

그러면 알고리즘을 버릴 필요가 없어.

1점씩 추가 와 IMSE + 예측오차 종료 는 그대로 두고, 그 사이의 계산만 줄이는 방향으로 가면 돼.

지금 네 코드에서 우리가 이미 확인한 건:

- 19개 GPR fitting: 약 8.4초 → 주범 같지는 않음
- 남은 강력한 후보: 65,536 후보 예측 + IMSE
- 이 중 특히 2000×1024 IMSE가 의심됨

그래서 내가 추천하는 순서는 아주 단순해.

지금 유지

```
80 DOE
↓
GPR
↓
IMSE로 1점
↓
CFD
↓
예측오차 확인
↓
다시 GPR/IMSE
```

대신 계산 최적화

1단계: ML_post_timing.py 로 실제 한 회차 시간을 측정.

예를 들어 결과가

GPR	8초
σ 계산	4초
IMSE	90초

면 바로 답이 나오지.

그럼 **IMSE** 계산만 빠르게 만들면 된다.

반대로

GPR	8초
σ 계산	60초
IMSE	10초

면 65,536 후보 쪽을 손봐야 하고.

그리고 중요한 건 **2,000 / 1,024**를 무작정 줄이지 않는 것이야.

예를 들어 현재:

2000×1024

를

1000×512

로 하면 계산 규모가 대략 1/4가 돼.

그런데 우리가 원하는 건:

"계산량은 줄이되, 다음에 선택되는 1점은 최대한 기존과 똑같이 만들기

야.

이건 충분히 테스트할 수 있어.

예를 들어 같은 데이터에서 현재 설정과 축소 설정으로 각각 다음 점을 뽑아보고, 선택된 점이 얼마나 달라지는지 비교하면 돼.

그래서 지금은 딱 이것만 하자

batch도 하지 말고, **LOOCV**도 하지 말고, 종료기준도 건드리지 말자.

현재 알고리즘 그대로 두고:

"한 점을 고르는 데 왜 오래 걸리는가?"

부터 잡자.

네가 `ML_post_timing.py` 로 적응샘플링 한 번 돌린 결과를 가져오면, 그 숫자 보고 내가 가장 영향 큰 한 군데만 먼저 줄이는 쪽으로 가자.

지금은 방법을 알아냈을 단계가 아니라 현재 방법의 계산비용을 다이어트할 단계야.

도대체 뭐가이렇게 오래걸리는거지

지금까지 나온 것만 보면 범인은 아직 특정되지 않았어. 그런데 후보는 거의 좁혀졌어.

현재 `ML.py` 의 해석 후 작업은 크게 4개야.

해석 완료

↓

① GPR 19개 학습 ≈ 8.4 초 ← 이미 측정됨

↓

② 65,536개 후보에 대해
17개 GPR σ 예측

↓

③ 그중 2,000개에 대해
17개 GPR $\times$ IMSE 계산

↓

④ 다음 점 결정

그래서 ①은 현재 숫자상 주범이 아니야.

내가 지금 가장 의심하는 건 ③ IMSE야.

특히 이 부분:

Python

실행됨

```
for name in active_objs:
    gpr, _use_log = models[name]
    red = _imse_reduction(gpr, cand_top, ref)
```

그리고 `_imse_reduction()` 안에서:

Python

실행됨

```
Kxc = k1(Xt, cand)
A = cho_solve((gpr.L_, True), Kxc)
```

```
chat = k1(ref, cand) - k1(Xt, ref).T @ A
```

를 **objective**마다 반복해.

현재 기본값이:

후보	2,000개
reference	1,024개
목적함수	최대 17개
학습 데이터	약 80~수백개

라서 상당히 큰 행렬 연산이 반복돼.

하지만 "**분명 IMSE 때문이다**"라고 지금 단정하면 안 돼.

네가 아직 `_gpr_suggest()` 전체 시간을 실제로 찍은 결과를 안 보여줬기 때문이야.

그래서 지금 제일 중요한 건

`ML_post_timing.py`로 **적응샘플링** 한 번 돌린 다음 이 부분만 보면 된다.

[GPR fitting]	?
[후보 생성 + 거리필터]	?
[1단계 sigma prediction]	?
[2단계 IMSE]	?
[_gpr_suggest 전체]	?
[해석 종료 → 다음점]	?

이걸 보면 바로 알 수 있어.

예를 들어:

GPR	8초
sigma	5초
IMSE	90초

→ **IMSE 범인**

또는

GPR	8초
sigma	70초
IMSE	5초

→ **65,536 후보 prediction이 범인**

또는

인데 실제로 해석 후 3분 기다린다

→ **ML.py**가 아니라 **ML.py**를 호출하는 바깥쪽 코드/프로그램에 병목이 있음

이렇게 딱 걸린다.

그러니까 지금은 숫자를 더 추측할 필요가 없어.

한 번 adaptive sampling이 실제로 실행된 로그를 보면 원인을 잡을 수 있어.

아니 이거 체감상 한 10~15분 잡아먹는거같음

아, **10~15분**이면 얘기가 완전히 달라져.

지금 우리가 측정한 **GPR 8.4초**는 거의 확실히 주범이 아니야. 10~15분을 8초로 설명할 수는 없으니까.

이 정도 시간이 걸린다면 내가 가장 먼저 의심하는 건 **IMSE** 계산 또는 그보다 더 바깥의 호출 구조야.

현재 코드에서 `_gpr_suggest()` 가 하는 일만 봐도:

```
GPR 19개 학습          ~ 8초
    ↓
65,536 후보 × 17 GPR 예측
    ↓
상위 2,000개 선정
    ↓
17개 GPR 각각에 대해 IMSE
    ↓
2,000 후보 × 1,024 reference
    ↓
다음 점 선택
```

이 중 IMSE는 단순히 `2000 × 1024` 숫자만 처리하는 게 아니라, 각 후보에 대해 공분산 행렬과 `cho_solve`, 행렬곱을 수행하고 있어.

그런데 **10~15분**이면 이것조차 실제로 얼마나 걸리는지 직접 찍어봐야 해.

그리고 한 가지 중요한 가능성이 있어:

ML.py 안에서 같은 계산이 여러 번 호출될 수도 있어

네 코드에 `_predict_point()` 가 있고:

Python

실행됨

```
def _predict_point(df, params):  
    models = _fit_models(df)
```

인데 `use_cache=False`라서 예측 캐시가 없으면 19개 GPR을 다시 처음부터 학습해.

정상적인 `get_next_params() → update_ml()` 흐름에서는 `_PENDING_PREDICTION`으로 이걸 피하도록 되어 있지만, 실제 프로그램의 호출 순서가 우리가 기대한 것과 다르면 이 경로가 다시 실행될 수 있어.

그러면 우리가 앞에서 짰은 8초 한 번이 아니라:

```
GPR 8초  
+  
다시 GPR 8초  
+  
IMSE  
+  
기타
```

가 될 수 있어.

그래서 지금은 추측을 멈추고 딱 한 번 확인해야 해

10~15분이 실제로 어디에서 발생하는지를 구간별로 찍어야 한다.

그리고 네가 이미 가지고 있는 `ML_post_timing.py`는 그 목적에 맞게 만들어놨으니까, 적응샘플링이 실제로 실행되는 한 회차에서 아래 로그를 한번만 가져오면 된다.

```
[GPR 학습 완료]  
[후보 생성 + 거리필터]  
[1단계 sigma prediction]  
[2단계 IMSE]  
[_gpr_suggest() 전체]  
[POST-CFD -> NEXT-CFD 전체 시간]
```

특히 나는 `IMSE` 숫자를 제일 보고 싶어.

만약:

```
GPR = 8초  
IMSE = 9분 30초
```

면 끝이야. 원인이 거의 확정돼.

그 경우에는 **IMSE 방식 자체를 버릴 필요 없이** `_imse_reduction()` 을 수학적으로 같은 결과에 가깝게 훨씬 싸게 계산하는 방향으로 손볼 수 있어.

반대로:

```
GPR = 8초  
IMSE = 20초  
전체 = 10분
```

이면 그 10분은 `ML.py` 의 다른 부분이나 바깥 해석 프로그램에서 쓰는 거라 `ML.py` 만 계속 뜯어 고쳐서는 해결이 안 돼.

그러니까 지금 상황을 한 문장으로 정리하면:

10~15분이면 GPR 19개가 문제가 아니라, 아직 측정하지 않은 다른 구간이 범인일 가능성이 매우 높다.

한 번 실제 adaptive 회차를 돌려서 타이밍 로그를 찍어보자. 그 결과만 있으면 이번에는 어디를 고쳐야 하는지 바로 좁힐 수 있어.

근데 그정도계산은 너가 가짜데이터로 실험해볼수있는거아님?
얼마나 걸리는ㄷ지

28s 동안 처리함

맞아. 그래서 방금 내가 네 코드의 핵심 계산을 가짜 데이터로 실제 돌려봤어.

조건을 네 코드와 거의 맞춰서:

- 학습 데이터 80점
- 입력변수 8개
- 목적함수 17개
- 후보 65,536개
- IMSE 후보 2,000개
- reference 1,024개

로 테스트했어.

결과는 이랬어.

17개 GPR 학습	약 2.35초
65,536개 × 17개 σ	약 3.49초
17개 IMSE 계산	약 1.28초

핵심 계산 합계	약 7.1초

즉 적어도 일반적인 Python/SciPy 환경에서 이 정도 규모의 IMSE가 10~15분 걸릴 구조는 아니야.

그리고 이건 꽤 중요한 결과야.

그러면 지금 상황은 이렇게 봐야 해

네가 실제로:

해석 끝 → 다음 해석 시작까지 10~15분

을 겪고 있다면, 우리가 지금까지 보고 있던 GPR/IMSE 계산 자체가 주범이 아닐 가능성이 매우 높아.

물론 네 PC와 내 실행환경은 CPU가 다르니까 정확한 초 단위 비교는 못 하지만, 1~10초대 규모의 연산이 600~900초로 늘어나는 정도의 차이는 정상적인 하드웨어 차이로 설명하기 어렵다.

오히려 의심해야 할 것

내가 지금 가장 의심하는 건 `ML.py` 가 같은 계산을 반복해서 호출하는 것 또는 `ML.py` 바깥쪽에 서 뭔가 기다리는 것이야.

특히 네 코드에는 이게 있어:

Python

실행됨

```
def _predict_point(df, params):  
    models = _fit_models(df)
```

그리고 여기서는:

Python

실행됨

```
_fit_models(df)
```

가 `use_cache=False` 라서 19개 GPR을 다시 최적화해.

정상 흐름에서는 `_PENDING_PREDICTION` 으로 이걸 피하도록 되어 있지만, 실제 해석 프로그램이 `update_ml()` / `get_next_params()` 를 어떻게 호출하는지에 따라 이 경로가 반복 실행될 가능성을 확인해야 해.

그래서 이제는 아주 확실하게 잡을 수 있어

다음 버전에서는 단순히 GPR 시간만 찍지 말고:

```
[1] update_ml 진입
[2] _predict_point 호출
[3] _fit_models 진입
[4] _fit_models 종료
[5] _gpr_suggest 진입
[6] 후보 생성
[7] sigma
[8] IMSE
[9] _gpr_suggest 종료
[10] get_next_params 종료
```

를 전부 시작/종료 timestamp로 찍어야 해.

그러면 실제로 10분이 어디에서 발생하는지 바로 잡힌다.

그리고 한 가지 더 중요한 발견

가짜 데이터 실험 결과를 보면 현재 2000 과 1024 를 무조건 줄여야 한다고 볼 근거도 없어졌어.

즉 지금은:

IMSE 숫자를 줄이는 것보다 먼저 실제 프로그램에서 10~15분이 어디에서 발생하는지 찾는 게 우선이야.

네가 말한 10~15분은 지금 상황에서 정상적인 **GPR+IMSE** 계산시간으로는 너무 크다.

다음에는 내가 네가 붙여준 `ML.py` 를 기준으로 "해석 완료 후 들어오는 순간부터 다음 해석점 반환까지" 모든 함수 호출 시간을 찍는 디버깅 버전을 만들어 주는 게 맞아.